# Course 1 Master Revision Guide: Generative AI & LangChain

## 1. Core Paradigms: GenAI, RAG & Agents

- GenAI Models: Predict and generate new text, images, code, or audio based on patterns.
- RAG (Retrieval Augmented Generation): Enhances LLMs by retrieving dynamic external facts at runtime. Reduces hallucinations without retraining model weights.
- Agentic AI: Systems that reason, plan, and autonomously execute multi-step tasks using external tools and APIs.

## 2. Prompt Engineering & Context Learning

- Instructions vs. Context: Instructions tell the AI what task to perform; Context provides background data.
- Zero-Shot: No examples provided; relies purely on pre-trained knowledge.
- One-Shot / Few-Shot: Providing 1 or a few demonstrations to guide output structure/style.
- Chain-of-Thought (CoT): Asking the model to explain its reasoning step-by-step.
- Self-Consistency: Sampling N independent CoT paths and taking a majority vote to eliminate logic glitches.

## 3. LangChain Expression Language (LCEL)

- The Pipe Syntax: chain = prompt | model | output_parser
- Why LCEL?: Replaces clunky, nested legacy classes (like LLMChain) with clean Unix-style pipelines.
- Type Coercion: Passing a Python function automatically wraps it as a RunnableLambda; passing a dict runs tasks concurrently.

## 4. System Design: RAG vs. Fine-Tuning

- RAG: Best for DYNAMIC, rapidly changing facts (e.g., news, company docs, stock prices). Zero model retraining cost.
- Fine-Tuning: Best for changing BEHAVIOR, tone, style, or custom syntax. Expensive and hard to update when facts change.
- Rule of Thumb: RAG for knowledge (facts); Fine-Tuning for style/behavior.

## 5. Cost & Latency Optimization (Start Big, Scale Small)

- Phase 1 (Quality Ceiling): Prototype your prompt on a large model (GPT-4o, Claude 3.5 Sonnet).
- Phase 2 (Downsize & Route): Evaluate the exact prompt on smaller 8B models (Llama-3-8B, GPT-4o-mini).
- Result: Slashes API costs by 90%+ and lowers latency while maintaining output quality.

## 6. ReAct Agent Framework (Reason -> Act -> Observe)

- The Agentic Loop: 1. Thought (Reason) -> 2. Action (Call Tool) -> 3. Observation (Read Tool Result) -> Repeat until goal met.
- Tools (@tool Decorator): The LLM reads the function's docstring as its instruction manual.
- Legacy Note: Lab 2 used AgentExecutor; in Course 7 you will build ReAct agents using modern LangGraph.

## 7. Vector Search & Advanced RAG (Small-to-Big Retrieval)

- Embeddings & Vector Space: Translating text into math vectors where similar meanings plot close together.
- RecursiveCharacterTextSplitter: Smart chunking that splits on paragraphs/sentences rather than blindly cutting text.
- ParentDocumentRetriever: Index tiny child chunks for search precision, but return large parent chunks for complete LLM context.

## 8. Model Architecture: Mixture of Experts (MoE)

- How MoE Works: Uses specialized sub-networks ('experts'). A router activates only 2 out of 8 experts per token (e.g., Mixtral 8x7B).
- Advantage: Gives 47B-level intelligence at 13B-level inference speed and cost.